

PARITYMEM: SEMANTIC COMPATIBILITY PREDICTION FOR STRUCTURED LLM INFERENCE

Authors to be confirmed

Affiliation and email pending author confirmation

ABSTRACT

Comparing backends for tool-using LLMs requires separating interface effects from model behavior: benign ID rewrites may look different, while locally valid calls can block continuation. PARITYMEM predicts compatibility and localizes source-supported consumer violations before outcome inspection. Pairs with validated compatibility and reachability are admitted; within-backend stability precedes canonical-action comparison. Across 354 version-pinned trace–model–backend cells, PARITYMEM accepts all 295 safe cells and detects all 59 blocked cells. A new adapter and contract reuse the frozen checking core on 144 trace–contract conditions (48 traces $\times$ 3 contracts), matching validation. Among 36 admitted backend pairs, five runs per backend yield 33 pairs with the same canonical action and 3 stable differences. Median recurring audit latency is 3.94 ms over 7,200 timed repetitions of 72 decisions, excluding contract specification and model inference. Together, these results make interface validity explicit, support reuse across source-specific contracts, and expose stable action differences while preserving permitted variation.

Index Terms— semantic compatibility, structured LLM inference, tool-using agents, inference backends, evaluation validity

1. INTRODUCTION

Reliable evaluation of structured LLM inference must distinguish model behavior from interface effects. Benchmarks such as τ -bench, ToolBench (introduced in ToolLLM), and AppWorld define tasks [1–3]. Model-specific dialog protocols structure tool interactions [4], while template mismatches can affect behavior [5]. Serving systems including vLLM and SGLang [6, 7] impose runtime acceptance conditions.

Figure 1 shows that consistent ID renaming can preserve an interaction despite different strings. Figure 2 shows that parsing, execution, and pairing can succeed while the next-history consumer rejects the turn. Literal equality flags the benign rewrite; local schema validity misses the continuation failure.

PARITYMEM answers three questions before attributing output differences to model behavior: (i) Is each trace contract-compatible despite benign representation changes? (ii) For a source-supported defect, which consumer obligation fails first and what consequence is demonstrated? (iii) If both sides are validated and reachable, do their stable canonical actions agree?

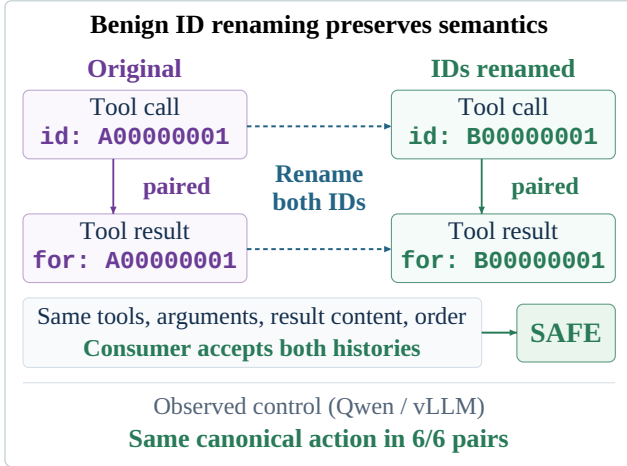


Fig. 1. Benign ID renaming preserves semantics. Jointly renaming a call ID and its result reference preserves pairing, content, order and consumer acceptance. All 6/6 Qwen/vLLM within-backend control pairs retain the same canonical action.

Tool Playgrounds evaluates tool interaction; WTU-EVAL evaluates tool-use necessity [8, 9]. AgentSuite, BenchGuard, and Auto Benchmark Audit [10–12] audit specifications, environments, ground truths and evaluators. AgentRx [13] localizes failure steps; HarnessFix [14]

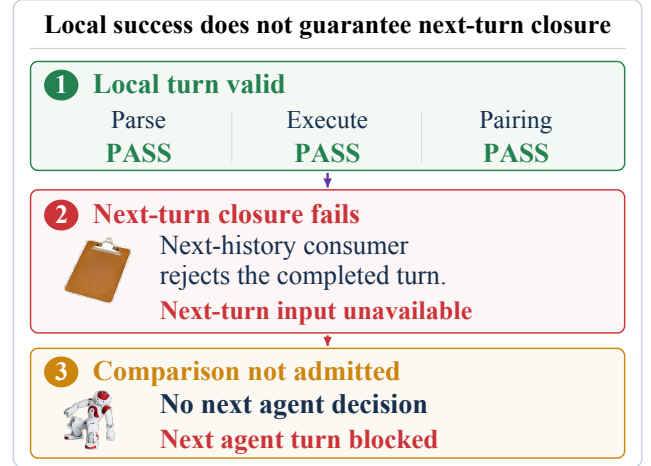


Fig. 2. Local success does not guarantee next-turn closure. Parsing, execution and pairing succeed, but the next-history consumer rejects the completed turn.

diagnoses and repairs harness flaws. PARITYMEM makes *comparison admissibility* explicit: source-grounded checks preserve permitted rewrites (Fig. 1) and diagnose consumer violations (Fig. 2), admitting validated, reachable pairs for stable action comparison.

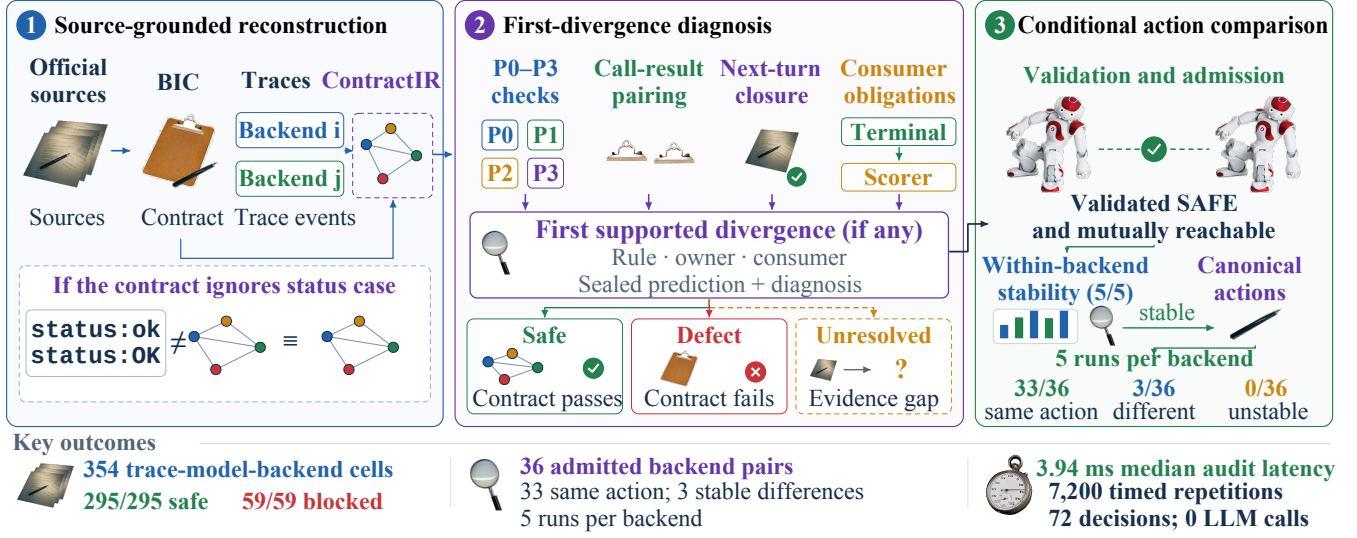


Fig. 3. PARITYMEM: source-grounded reconstruction, first-divergence diagnosis, and conditional action comparison. Predictions are sealed; officially validated SAFE and consumer reachability admit pairs. Within-backend 5/5 stability precedes canonical-action comparison. Bottom: 354 trace-model-backend cells, 36 admitted backend pairs, and recurring audit latency from 7,200 timed repetitions over 72 decisions.

2. COMPATIBILITY-FIRST COMPARISON

PARITYMEM supports comparison admission with compatibility verdicts and source-linked diagnoses (Fig. 3). For each supported defect, the record identifies the violated predicate, its owner and consumer, the first divergence, and the strongest demonstrated consequence.

2.1. Source-grounded reconstruction

For interaction x and backend b , authoritative sources $\mathcal{S}_x$ define a behavioral interface contract (BIC) C_x governing the structured trace $T_{b,x}$:

$$C_x = \text{BIC}(\mathcal{S}_x), \quad I_{b,x} = \text{ContractIR}(T_{b,x}; C_x). \quad (1)$$

Each obligation specifies its source and version, owner, consumer, and acceptance condition. Source contracts supply integration-specific domains and predicates; record adapters populate shared CONTRACTIR fields. Pairing, projection checks, and source-attributed diagnosis operate over those fields. Human review confirms source authority and precedence, enabling repeatable automatic checks across histories.

CONTRACTIR retains raw and semantic forms of typed messages, calls, arguments, results, transitions, and identifier/scorer bindings, with order, owner, provenance, and consumer. Transport-ID renaming preserves meaning when it retains the call-result bijection and consumer acceptance. For a contract-relevant projection Π_{C_x} and next consumer N_{C_x} , a benign change ϕ satisfies

$$\Pi_{C_x}(T) = \Pi_{C_x}(\phi(T)), \quad \{T, \phi(T)\} \subseteq \text{Dom}(N_{C_x}). \quad (2)$$

Equation (2) defines admissible representation freedom; diagnostic probes restore a target acceptance condition while preserving the specified semantic projection and non-target obligations.

The audited SGLang-Mistral path illustrates identifier-domain non-closure and source ownership: SGLang emits a 29-character action ID, whereas the exact Mistral template requires a nine-character alphanumeric result-binding ID [15]. The ID domain belongs to the consumer contract; the binding and order checks remain shared. An admissible remap is collision-safe and injective, preserving call-result correspondence, tools, arguments, order, and state.

2.2. First-divergence diagnosis

Cross-turn compatibility requires the next consumer to accept the completed history. On $I_{b,x}$, P0 checks representability; P1 checks model- and benchmark-visible meaning; P2 checks actions, authoritative state transitions, and continuation; P3 checks terminal state and scorer-facing bindings. These projections check distinct obligations.

Pairing verifies unique call-result correspondence across transport, execution, and model-visible IDs. For the next-history serializer H_x , closure checks $T_{b,x} \in \text{Dom}(H_x)$ and preserves the relevant CONTRACTIR projection on the round trip. The frozen contract, versions, and recorded path define the audit scope.

The checker attributes violations to source predicates and returns the first source-supported divergence in recorded interaction/consumer order, not P0-P3 evaluation order. Manifestations of the same predicate share one root. The sealed prediction is $\hat{V}_{b,x} = J(C_x, I_{b,x}) \in \{\text{SAFE}, \text{DEFECT}, \text{UNRESOLVED}\}$: SAFE requires all applicable obligations to pass with sufficient evidence; DEFECT identifies a reproducible source-supported violation; UNRESOLVED preserves uncertainty in authority or applicability. Consequence R2 records changed model-visible meaning, R3 a demonstrated pairing, state, or continuation effect, and R4 an observed scorer/task effect. Verdict and reachability remain separate: altered content may still allow execution to continue.

The case in Fig. 2 contains three paired calls: `get_order`, `get_policies`, and `process_refund`, returning order, policy, and preview results. Parsing, pairing, and current-turn execution succeed, but the Llama next-history cardinality guard rejects the completed turn (R3). A projection-preserving probe restores reachability from 0/3 to 3/3, with actual continuation in 3/3; these fractions count per-root probe outcomes. Prediction targets next-history acceptance from the completed turn before inspecting that consumer’s validation outcome.

Figure 1 preserves both the semantic projection and consumer acceptance in Eq. (2); Figure 2 fails next-turn closure despite local success.

2.3. Conditional action comparison

Compatible interfaces can produce different actions. Let $A_{b,x}$ denote status after official validation and $r_{b,x} = 1$ required-consumer reachability. Validation updates $A_{b,x}$, not the sealed prediction $\hat{V}_{b,x}$. A pair is admitted when

$$E_{i,j,x} = 1[A_{i,x} = A_{j,x} = \text{SAFE} \wedge r_{i,x} = r_{j,x} = 1]. \quad (3)$$

Admitted pairs require 5/5 stability within each backend before action comparison. Identifiers are retained for contract and pairing checks; after admission, canonical actions omit permitted transport IDs and encode kind and ordered (tool, argument) entries. We decode JSON argument strings and sort object keys, preserving array order and string values, including code. Nonempty outputs without accepted calls map to NOTOOL; parser errors and empty outputs are invalid. Execution equivalence and task scores remain separate endpoints.

Algorithm 1. Compatibility-first comparison

Input: confirmed contracts and recorded traces.

1. Lift each trace into its contract-conditioned CONTRACTIR.
2. Check P0–P3, pairing, and history-closure obligations.
3. Select the first source-supported violation, if any.
4. Seal each prediction and its source-linked diagnostic record.
5. Obtain official validation and required-consumer reachability.
6. Admit pairs satisfying Eq. (3).
7. Test five-run stability; compare the stable canonical modes.

3. EXPERIMENTS

3.1. Experimental setup and validation protocol

A matrix cell specifies a benchmark, trace, model/revision, and backend/version. Mechanism probes count distinct roots; an adapter condition is one trace–contract combination. An admitted backend pair fixes benchmark, trace and model revision, with both paths validated SAFE and required consumers reachable. Denominators are never pooled.

Policies are frozen Qwen3-14B, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3 revisions [4, 16, 17]. They consume official histories and tool schemas from τ^3 -bench and AppWorld [1, 3, 18] via SGLang or vLLM. Original backend conditions fix weights, tokenizer, model-specific template, semantic history, tool schemas and generation parameters (temperature zero). The companion’s START_HERE.md lists exact revisions. PARITYMEM invokes no LLM; inference is reserved for behavioral validation.

Table 1 separates sealed Full, post-hoc compatibility baselines and aggregate-derived components; adjacent systems are discussed by task and output. On frozen record pairs, raw equality retains IDs; schema checks types/tool names; normalization decodes JSON and removes IDs without reordering. Rules use no outcome labels or PARITYMEM logic; 1,062 per-cell baseline outputs are scored against frozen official outcomes.

In controlled suites, PARITYMEM preserves 280/280 clean replays, detects 120/120 mutations, satisfies 100/100 safe and 100/100 unsafe metamorphic relations, and correctly classifies 300/300 blind holdout cases, including the 120/120 novel-composition subset. The source-derived holdout generator/gold program imports no PARITYMEM prediction logic, and the prediction runner has no gold access. An independent programmatic adjudicator, with no gold access or checker imports, agrees on class, consequence, and first divergence for 60/60 sampled cases. The companion provides the frozen core and recorded inputs for these controls.

3.2. Compatibility accuracy and diagnostic value

The original matrix uses SGLang 0.5.6.post2 and vLLM 0.23.0. Of 60 sampled histories, 59 are eligible across three models and two backends (354 cells); one AppWorld history is inapplicable in all six settings. Full predictions precede official parser/template outcomes. Full PARITYMEM matches 354/354 binary verdicts, 59/59 R3 labels, and 354/354 first-divergence labels. All 59 blocks reflect identifier-domain non-closure; this matrix has no R2 or unresolved cases.

Table 1. Frozen matrix: 354 cells (295 safe, 59 blocked). Full: sealed per-cell predictions; baselines: post-hoc frozen records. ‡: historical component summaries from stratum counts. Consequence scoring: 59 R3 blocks (demonstrated pairing, state or continuation effects). First divergence: 295 NONE; 59 identifier-domain non-closure. N/A: no emitted per-cell label.

Method / variant	Binary verdict correct	Safe cells falsely flagged	Blocked cells detected	Consequence label correct (R3)	First-divergence correct
<i>Offline record baselines (post hoc)</i>					
Raw equality	59/354	295/295	59/59	N/A	N/A
Strict schema	295/354	0/295	0/59	N/A	N/A
Generic normalization	295/354	0/295	0/59	N/A	N/A
<i>Sealed per-cell audit</i>					
Full PARITYMEM	354/354	0/295	59/59	59/59	354/354
<i>Aggregate-derived component summaries</i>					
No pairing semantics‡	59/354	295/295	59/59	N/A	N/A
No history closure‡	295/354	0/295	0/59	N/A	N/A
P0/P1 only‡	295/354	0/295	0/59	N/A	N/A

Balanced accuracy averages safe acceptance ($1 - \text{FP}/295$) and blocked-cell recall ($\text{TP}/59$). Full PARITYMEM achieves **100%**: 0/295 safe cells falsely flagged and 59/59 blocked cells detected. The three offline baselines and three historical component summaries each yield 50%; Full also supplies consequence and first-divergence labels.

A non-random audit of ten released integrations confirms four roots in five integrations: next-turn, identifier-domain, and observation-role non-closure (all R3), plus model-visible content erasure (R2). P0/P1: 4/4 root detection, 1/4 R2/R3 attribution; full P0–P3: 4/4 for both (Table 2).

Table 2. Four-root probes: coverage and consequence attribution. Every fraction uses four roots, separately from the 354-cell matrix.

Closure variant	Root recall	Consequence evidence
Parse-only	1/4	Partial coverage
Structural	3/4	Partial coverage
Semantic	4/4	R3 underclassified
Full execution	4/4	4/4 correct
Projection variant	Root detection	R2/R3 correct
P0/P1 only	4/4	1/4
Full P0–P3	4/4	4/4

Literal-ID matching yields 36 safe false alarms; order-only pairing misses identifier-domain non-closure. Probes preserve specified semantics and non-target obligations, restoring next-turn reachability from 0/3 to 3/3 for each non-closure root; the next-turn case continues in 3/3. Scanning 250 official histories activates next-turn and identifier-domain non-closure, and model-visible content erasure.

A frozen Qwen/vLLM within-backend control tests six ID-remap pairs. All 60 calls complete, every condition is 5/5 stable, and all 6/6 pairs preserve both CONTRACTIR equivalence and the modal canonical action. Permitted rewrites can preserve canonical actions as well as interface semantics.

3.3. Frozen-core reuse and version validation

A new integration-specific adapter and contract reuse the frozen checking core on 144 controlled trace–contract conditions (48 traces, each under three applicable contract variants). Verdict, consequence and first-divergence labels match validation in all 144: 16/16 safe conditions are accepted and 128/128 defects detected. Name aliases preserve 144/144 decisions; 48/48 contract-change contrasts, involving 24 traces, match expected responses. These controls verify frozen-core reuse across source-specific contracts; version tests use their respective source profiles. Held-out contract transfer yields full BICs for τ^3 -bench and ToolSandbox [19], and a partial AppWorld BIC (9/10 fields). AppWorld transfer evidence remains contract-level.

SGLang 0.5.16, absent from method construction, was selected and sealed before outcome inspection. The prediction runner has no outcome labels; the outcome runner imports no PARITYMEM decision logic. Source-only predictions match 177/177 official outcomes: 118 safe and 59 identifier-domain non-closure manifestations. Separate input, sealed-prediction, and consumer-outcome records make reuse beyond construction auditable.

On vLLM 0.11.0, 165/177 sealed SAFE predictions agree with official outcomes. The remaining 12 Llama cases—ten AppWorld and two τ^3 —encounter official parser rejection. Their sealed predictions remain unchanged; validation records their final status as UNSOLVED. All 177 stay in the agreement denominator, while Eq. (3) excludes those 12 from behavioral comparison. This validation record (Table 3) links admission to observed consumer acceptance.

Table 3. Sealed predictions and official validation. S/D lists predicted Safe and Defect counts. Final S/B/U lists Safe, Blocked, and Unsolved outcomes; validated defects in these matrices are blocking.

Setting	Pred. S/D	Agreement	Final S/B/U
Original matrix	295/59	354/354	295/59/0
SGLang 0.5.16	118/59	177/177	118/59/0
vLLM 0.11.0	177/0	165/177	165/0/12

3.4. Action comparison and audit cost

Using SGLang 0.5.6.post2 and vLLM 0.23.0, the prospective panel admits 36 of 80 mutually reachable candidates, balanced over Qwen/Llama and both benchmarks without prior agreement labels. Five runs per backend, hash-assigned across two independently launched server epochs, yield $36 \times 2 \times 5 = 360$ calls. All 72 backend conditions are 5/5 stable. Canonical actions agree in 33/36 pairs and stably differ in 3/36; none is unstable. The 288/288 same-process and 432/432 cross-restart agreements are within-condition comparisons, not additional calls.

Three Llama pairs show stable canonical-action differences (5/5 per condition). Both AppWorld cases select `python.exec`: trace 036 differs in the Gmail app-name argument to continuation-token lookup; trace 088 uses environment-result versus environment access. On τ^3 , SGLang selects no tool while vLLM calls `get_order_details` for order 12345. Together, the remap control and admitted-pair results show how PARITYMEM preserves permitted representation changes while exposing stable canonical-action differences.

The checker invokes no LLM. On a separate 72-decision matrix subset, 20 warmups and 100 timed repetitions per decision yield 7,200 records, all matching frozen verdicts; the companion provides the timing records. Independently timed recurring audit latency T_{total} has median 3.94 ms and 95th percentile (p95) 5.48 ms, from JSON materialization through CONTRACTIR construction to first-divergence selection. Source inspection, human BIC specification, tokenizer loading and model inference are excluded. Fresh-process 100-decision batches yield median/p95 increases of 0.38/0.92 MiB in resident set size (RSS, process-resident host memory). On Python 3.11.15 pinned to one Intel Xeon Silver 4214 logical CPU, median latency rises from 3.57 to 36.26 ms over the tested 1–32-call range.

4. DISCUSSION AND PRACTICAL USE

For backend updates, versioned contracts and sealed input, prediction, and validation records make integration decisions traceable. Once a contract is confirmed, repeated checks require no LLM calls (Algorithm 1). PARITYMEM complements benchmark auditing, trajectory diagnosis, harness repair and observability/recovery [10–14, 20] by establishing when stable cross-backend actions can be compared. Execution and scoring assess task quality.

5. CONCLUSION

PARITYMEM preserves permitted variation, localizes the first source-supported consumer violation, and compares canonical actions on admitted, stable paths. The 144 trace–contract conditions demonstrate frozen-core reuse, complementing the matrix, mechanism roots, version tests and 36 backend pairs at millisecond-scale recurring audit latency.

Acknowledgment. OpenAI ChatGPT assisted wording and organization of the abstract and Sections 1–5, figure-layout code, and companion code/documentation. Model experiments are unchanged.

Photo credits (Figs. 2–3; cropped, masked, and arranged). Wood-clipboard: Evan-Amos; Apollo 17 Earth: NASA (public domain). Magnifying glass: Tomomarusan (CC BY 2.5). Legal Contract & Signature: Blogtrepreneur (CC BY 2.0). NAO Robot: Philippe Dureuil/TOMA, released by Softbank Robotics Europe (CC BY-SA 4.0); Stopwatch: Ansgar Koreng / CC BY-SA 4.0.

6. REFERENCES

- [1] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=roNSXZpUDN>
- [2] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=dHng200Jjr>
- [3] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 16 022–16 076. [Online]. Available: <https://aclanthology.org/2024.acl-long.850/>
- [4] A. Grattafiori *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024, version 3. [Online]. Available: <https://arxiv.org/abs/2407.21783v3>
- [5] F. Jiang, Z. Xu, L. Niu, B. Y. Lin, and R. Poovendran, “ChatBug: A common vulnerability of aligned LLMs induced by chat templates,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 26, pp. 27 347–27 355, 2025. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/34945>
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, 2023, pp. 611–626. [Online]. Available: <https://dl.acm.org/doi/10.1145/3600006.3613165>
- [7] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
- [8] Z. Dong, R. Gong, Y. Yong, S. Wu, Y. Yao, S.-L. Chen, and X.-C. Yin, “Tool playgrounds: A comprehensive and analyzable benchmark for LLM tool invocation,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. IEEE, 2025, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/icassp49660.2025.10890828>
- [9] J. Liu, K. Ning, Y. Su, W. Han, J. Xu, and Y. Zhang, “WTU-EVAL: A whether-or-not tool usage evaluation benchmark for large language models,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. IEEE, 2025, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/icassp49660.2025.10889153>
- [10] H. Suh, B. Ji, S. Lee, R. Khare, B. Khan, H. Kim, T. Zhang, V. K. Srinivasan, P. Belcak, S. Diao, P. Molchanov, Y. C. Lin, and Z. Dong, “AgentSuite: Toward more reliable agent evaluation with a component-based benchmark auditing pipeline,” in *International Conference on Machine Learning*, 2026. [Online]. Available: <https://openreview.net/forum?id=2Exmr1eIKZ>
- [11] X. Tu, T. Wang, Y. Lu, K. Huang, Y. Qu, and S. Mostafavi, “BenchGuard: Who guards the benchmarks? Automated auditing of LLM agent benchmarks,” *arXiv preprint arXiv:2604.24955*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.24955>
- [12] J. Wang, F. Bianchi, S. Zhu, F. Nie, Y. Kwon, B. Dhingra, and J. Zou, “Automated benchmark auditing for AI agents and large language models,” *arXiv preprint arXiv:2605.26079*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26079>
- [13] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, “AgentRx: Diagnosing AI agent failures from execution trajectories,” *arXiv preprint arXiv:2602.02475*, 2026, version 2; accepted to Findings of EMNLP 2026. [Online]. Available: <https://arxiv.org/abs/2602.02475v2>
- [14] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, and Q. Wang, “From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws,” *arXiv preprint arXiv:2606.06324*, 2026, version 2. [Online]. Available: <https://arxiv.org/abs/2606.06324v2>
- [15] Mistral AI, “Model card for Mistral-7B-Instruct-v0.3,” Official model card, Hugging Face, accessed 2026-09-06. [Online]. Available: <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>
- [16] A. Yang *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [17] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [18] Sierra Research, “ τ^3 -bench software,” Official software repository, 2026, frozen revision 79975ac5741e23fbb1d2ac44262d62398a6d87bd; accessed 2026-08-16. [Online]. Available: <https://github.com/sierra-research/tau2-bench/tree/79975ac5741e23fbb1d2ac44262d62398a6d87bd>
- [19] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, “ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities,” in *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 2025, pp. 1160–1183. [Online]. Available: <https://aclanthology.org/2025.findings-naacl.65/>
- [20] K. Zhu, X. Ye, Z. Han, Y. Zhao, B. Li, W. Zhang, M. Tian, X. Tang, P. Lu, J. Zou, J. You, and H. Ji, “AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents,” *arXiv preprint arXiv:2607.18754*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.18754>